

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M. SnehaLatha	Reg. No.:	192425092
Section / Batch:	2024-AI&ML	Date:	

Code of Conduct

I M. Sneha(192425092) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. Sneha _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	3	3	3	3	2	15	87
2	4	4	4	4	4	20	
3	5	5	5	5	5	25	
4	3	3	3	3	2	15	
5	3	2	3	2	2	12	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 – Hidden Markov Model & Viterbi Algorithm for POS Tagging

1. Splitting Sentences into Words and POS Tags

Each training sentence is separated into a list of (word, tag) pairs by splitting on the '/' symbol.

The resulting tagged corpus is:

Sentence #	Word/Tag Sequence
1	The/DT boy/NN eats/VBZ rice/NN
2	The/DT girl/NN drinks/VBZ milk/NN
3	A/DT cat/NN drinks/VBZ milk/NN
4	The/DT dog/NN chases/VBZ cat/NN
5	A/DT teacher/NN teaches/VBZ students/NNS
6	Students/NNS study/VBP English/NN
7	Birds/NNS fly/VBP high/RB
8	Children/NNS play/VBP games/NNS

Python code:

```
corpus = [
    "The/DT boy/NN eats/VBZ rice/NN",
    "The/DT girl/NN drinks/VBZ milk/NN",
    "A/DT cat/NN drinks/VBZ milk/NN",
    "The/DT dog/NN chases/VBZ cat/NN",
    "A/DT teacher/NN teaches/VBZ students/NNS",
    "Students/NNS study/VBP English/NN",
    "Birds/NNS fly/VBP high/RB",
    "Children/NNS play/VBP games/NNS",
]

tagged_sentences = []
for sent in corpus:
    pairs = []
    for token in sent.split():
        word, tag = token.rsplit("/", 1)
        pairs.append((word.lower(), tag))
    tagged_sentences.append(pairs)
```

2. Emission Probability

Emission probability is $P(\text{word} \mid \text{tag}) = \text{count}(\text{word}, \text{tag}) / \text{count}(\text{tag})$. Counting each word against its tag over the whole corpus gives:

Tag frequency (denominator)

Tag	Count
DT	5
NN	10
VBZ	5
NNS	5
VBP	3
RB	1

Emission probabilities

Tag	Word : $P(\text{word} \text{tag})$
DT	the : $3/5 = 0.60$ a : $2/5 = 0.40$
NN	boy: $1/10=0.10$ rice: $1/10=0.10$ girl: $1/10=0.10$ milk: $2/10=0.20$ cat: $2/10=0.20$ dog: $1/10=0.10$ teacher: $1/10=0.10$ english: $1/10=0.10$
VBZ	eats: $1/5=0.20$ drinks: $2/5=0.40$ chases: $1/5=0.20$ teaches: $1/5=0.20$
NNS	students: $2/5=0.40$ birds: $1/5=0.20$ children: $1/5=0.20$ games: $1/5=0.20$
VBP	study: $1/3=0.33$ fly: $1/3=0.33$ play: $1/3=0.33$
RB	high: $1/1=1.00$

Python code:

```
from collections import defaultdict

emission_counts = defaultdict(lambda: defaultdict(int))
tag_counts = defaultdict(int)

for sent in tagged_sentences:
    for word, tag in sent:
        emission_counts[tag][word] += 1
```

```

tag_counts[tag] += 1

emission_prob = defaultdict(dict)
for tag in emission_counts:
    for word in emission_counts[tag]:
        emission_prob[tag][word] = emission_counts[tag][word] /
tag_counts[tag]

```

3. Transition Probability

Transition probability is $P(\text{tag}_i | \text{tag}_{i-1}) = \text{count}(\text{tag}_{i-1} \rightarrow \text{tag}_i) / \text{count}(\text{tag}_{i-1})$. Sentence-initial tags additionally define the start probability, $P(\text{tag} | \text{START}) = \text{count}(\text{tag as first tag}) / \text{total sentences}$.

Start probabilities

Tag	P(tag START)
DT	5/8 = 0.625
NNS	3/8 = 0.375

Transition probabilities

From → To	Count	Probability
DT → NN	5/5	1.00
NN → VBZ	5/5	1.00
VBZ → NN	4/5	0.80
VBZ → NNS	1/5	0.20
NNS → VBP	3/3	1.00
VBP → NN	1/3	0.33
VBP → RB	1/3	0.33
VBP → NNS	1/3	0.33

Python code:

```

transition_counts = defaultdict(lambda: defaultdict(int))

```

```

from_tag_totals = defaultdict(int)
start_counts = defaultdict(int)

for sent in tagged_sentences:
    tags = [t for _, t in sent]
    start_counts[tags[0]] += 1
    for i in range(len(tags) - 1):
        transition_counts[tags[i]][tags[i+1]] += 1
        from_tag_totals[tags[i]] += 1

num_sentences = len(tagged_sentences)
start_prob = {t: c / num_sentences for t, c in start_counts.items()}

transition_prob = defaultdict(dict)
for t1 in transition_counts:
    for t2 in transition_counts[t1]:
        transition_prob[t1][t2] = transition_counts[t1][t2] /
from_tag_totals[t1]

```

4. Recreating the HMM with Python Dictionaries

The complete Hidden Markov Model is represented purely with nested dictionaries (no external libraries):

```

states = ["DT", "NN", "VBZ", "NNS", "VBP", "RB"]

start_prob = {"DT": 0.625, "NNS": 0.375}

transition_prob = {
    "DT": {"NN": 1.0},
    "NN": {"VBZ": 1.0},
    "VBZ": {"NN": 0.8, "NNS": 0.2},
    "NNS": {"VBP": 1.0},
    "VBP": {"NN": 1/3, "RB": 1/3, "NNS": 1/3},
    "RB": {}
}

emission_prob = {
    "DT": {"the": 0.6, "a": 0.4},
    "NN": {"boy": 0.1, "rice": 0.1, "girl": 0.1, "milk": 0.2,
           "cat": 0.2, "dog": 0.1, "teacher": 0.1, "english": 0.1},
    "VBZ": {"eats": 0.2, "drinks": 0.4, "chases": 0.2, "teaches":
0.2},

```

```

    "NNS": {"students": 0.4, "birds": 0.2, "children": 0.2, "games":
0.2},
    "VBP": {"study": 1/3, "fly": 1/3, "play": 1/3},
    "RB": {"high": 1.0}
}

```

5. Viterbi Algorithm (No Built-in HMM/NLP Libraries)

The Viterbi algorithm fills a dynamic-programming table $V[\text{tag}][t]$ holding the highest probability of any tag path ending in that tag at position t , together with a backpointer table for path recovery.

```

def viterbi(words, states, start_prob, transition_prob,
emission_prob):
    V = [{}]
    backpointer = [{}]

    # Initialisation (t = 0)
    for s in states:
        emit = emission_prob.get(s, {}).get(words[0], 0.0)
        V[0][s] = start_prob.get(s, 0.0) * emit
        backpointer[0][s] = None

    # Recursion (t = 1 .. n-1)
    for t in range(1, len(words)):
        V.append({})
        backpointer.append({})
        for s in states:
            emit = emission_prob.get(s, {}).get(words[t], 0.0)
            best_prob, best_prev = 0.0, None
            for prev in states:
                trans = transition_prob.get(prev, {}).get(s, 0.0)
                prob = V[t-1][prev] * trans * emit
                if prob > best_prob:
                    best_prob, best_prev = prob, prev
            V[t][s] = best_prob
            backpointer[t][s] = best_prev

    # Termination: pick best final state
    last_t = len(words) - 1
    best_final_state = max(V[last_t], key=V[last_t].get)
    best_prob = V[last_t][best_final_state]

```



```
# Backtrack the best path
path = [best_final_state]
for t in range(last_t, 0, -1):
    path.insert(0, backpointer[t][path[0]])

return path, best_prob, V
```

6. Predicting the POS Tags for “The cat drinks milk”

Input words (lower-cased): the, cat, drinks, milk. Since every one of these words was seen with only a single tag during training, each Viterbi column has exactly one non-zero cell, so the dynamic-programming trace below shows the full computation step by step.

Step-by-step Viterbi trace

t	Word	Best Tag	Calculation	V(t)
1	the	DT	$\text{start}(\text{DT}) \times P(\text{the} \text{DT}) = 0.625 \times 0.60$	0.3750
2	cat	NN	$V1(\text{DT}) \times P(\text{NN} \text{DT}) \times P(\text{cat} \text{NN}) = 0.375 \times 1.0 \times 0.20$	0.0750
3	drinks	VBZ	$V2(\text{NN}) \times P(\text{VBZ} \text{NN}) \times P(\text{drinks} \text{VBZ}) = 0.075 \times 1.0 \times 0.40$	0.0300
4	milk	NN	$V3(\text{VBZ}) \times P(\text{NN} \text{VBZ}) \times P(\text{milk} \text{NN}) = 0.03 \times 0.80 \times 0.20$	0.0048

Backtracking from the highest-probability final state (NN, $V = 0.0048$) through the stored backpointers $\text{DT} \rightarrow \text{NN} \rightarrow \text{VBZ} \rightarrow \text{NN}$ gives the predicted tag sequence:

The/DT cat/NN drinks/VBZ milk/NN

This matches the pattern of the closest training sentence (“A/DT cat/NN drinks/VBZ milk/NN”), confirming that the HMM and Viterbi implementation generalise correctly to the new input.

Overall path probability = 0.0048.

Driver code

```
sentence = "The cat drinks milk".lower().split()
states = ["DT", "NN", "VBZ", "NNS", "VBP", "RB"]
```

```

best_path, best_prob, V = viterbi(sentence, states, start_prob,
                                  transition_prob, emission_prob)

print("Predicted POS tags:")
for word, tag in zip(sentence, best_path):
    print(f"{word}/{tag}", end=" ")
print(f"\nPath probability: {best_prob}")

# Output:
# Predicted POS tags:
# the/DT cat/NN drinks/VBZ milk/NN
# Path probability: 0.0048

```

Conclusion

Using only Python dictionaries, loops and basic control structures, the emission and transition probabilities were computed directly from the tagged training corpus, the HMM was represented as nested dictionaries, and a from-scratch Viterbi algorithm correctly decoded the unseen sentence “The cat drinks milk” as DT NN VBZ NN, with no reliance on any built-in NLP or HMM library.